التاسك الشامل – بايثون

مشكلة: فاتورة المقصف

الاسم: تميم تركي الحارثي

الكلاس: _____

المادة: بايثون

التاريخ: 1 نوفمبر 2026

هذا الملف فيه الأجزاء المكتوبة (أ، ب، ج، د، و، ز + السؤال الأخير).
الجزء هـ (البرنامج) في ملف py . منفصل.

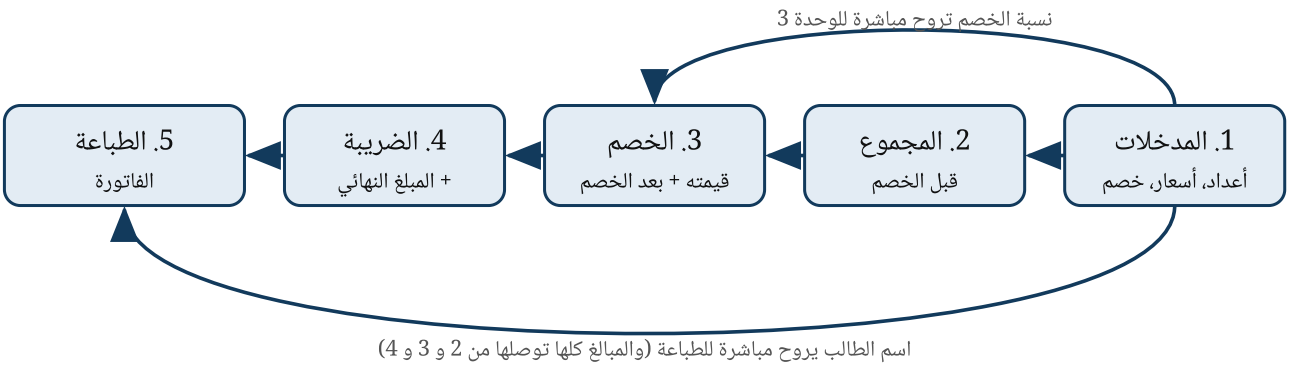
الجزء أ: التجزئة

1) المشاكل الفرعية

قسمت مشكلة الفاتورة إلى خمس مشاكل فرعية. كل وحدة لها شغلة وحدة بس:

#	اسم الوحدة	المدخلات	المخرجات
1	قراءة المدخلات	اللي يكتبه المستخدم: اسم الطالب، عدد السندويتشات وسعرها، عدد العصائر وسعرها، نسبة الخصم	متغيرات جاهزة بالنوع الصح (أعداد `int` وأسعار `float`)
2	حساب المجموع	عدد وسعر السندويتشات، عدد وسعر العصائر	المجموع قبل الخصم
3	حساب الخصم	المجموع قبل الخصم، نسبة الخصم	قيمة الخصم، والمجموع بعد الخصم
4	حساب الضريبة والنهائي	المجموع بعد الخصم، نسبة الضريبة 15%	قيمة الضريبة، والمبلغ النهائي
5	طباعة الفاتورة	اسم الطالب وكل المبالغ اللي فوق	فاتورة مرتبة، كل مبلغ برقمين بعد الفاصلة ومعه «ريال / SAR»

2) مين يعتمد على مين



السهم معناه «يعتمد على»: مثلا الضريبة (4) ما تنحسب إلا بعد ما يطلع المجموع بعد الخصم (3).

3) هل تجزئتي صحيحة؟

- **غطت كل المشكلة؟** نعم. كل شي مطلوب بالفاتورة (المجموع، الخصم، بعد الخصم، الضريبة، النهائي، الطباعة) له وحدة تطلعه، والمدخلات كلها لها وحدة تقراها.
- **فيه جزئين يسوون نفس الشي؟** لا. كل وحدة لها شغلة مختلفة: وحدة تقرا، ووحدة تجمع، ووحدة تخصم، ووحدة تضيف الضريبة، ووحدة تطبع. ما فيه حساب يتكرر في وحدتين.

- **كل جزء أقدر أختبره لحاله؟** نعم. مثلا أقدر أعطي وحدة الخصم مجموع 15 وخصم 20% وأتأكد إنها تطلع 3.00 و 12.00 بدون ما أحتاج باقي البرنامج. ووحدة الضريبة أعطيها 12.00 وأتأكد إنها تطلع 1.80.

4) حركات التفكير الحاسوبي الأربع من المشكلة

الحركة	مثال من مشكلة الفاتورة
التجزئة	قسمت الفاتورة الكبيرة إلى 5 مشاكل صغيرة (مدخلات، مجموع، خصم، ضريبة، طباعة).
التعرف على الأنماط	حساب السندويتشات وحساب العصائر نفس النمط بالضبط: **العدد** × **السعر**. وكمان الخصم والضريبة نفس النمط: **المبلغ** × **(النسبة** ÷ **100)**.
التجريد	تجاهلت التفاصيل اللي ما تأثر على الحساب، مثل نوع السندويتش أو نكهة العصير أو وقت الطلب. اللي يهمني بس: العدد والسعر.
تصميم الخوارزمية	كتبت خطوات مرتبة: اقرا المدخلات ← احسب المجموع ← احسب الخصم ← احسب الضريبة ← اطبع (موجودة بالجزء ب).

الجزء ب: الخوارزمية والشيفرة الزائفة

1 و 2) الخوارزمية

```
START
  // Sub-problem 1: read inputs
  INPUT student_name
  INPUT sandwich_count          // whole number
  INPUT sandwich_price          // SAR
  INPUT juice_count             // whole number
  INPUT juice_price             // SAR
  INPUT discount_percent        // e.g. 20 means 20%
  SET vat_percent = 15

  // Sub-problem 2: subtotal
  SET sandwiches_cost = sandwich_count * sandwich_price
  SET juices_cost     = juice_count * juice_price
  SET subtotal        = sandwiches_cost + juices_cost

  // Sub-problem 3: discount  (decision from Part C)
  IF discount_percent > 50 THEN
      OUTPUT "Warning: discount is more than 50%"
  END IF
  SET discount_amount = subtotal * (discount_percent / 100)
  SET after_discount  = subtotal - discount_amount

  // Sub-problem 4: VAT and final total
  SET vat_amount  = after_discount * (vat_percent / 100)
  SET final_total = after_discount + vat_amount

  // Sub-problem 5: print (every amount with 2 decimals + "SAR")
  OUTPUT student_name
  OUTPUT subtotal, discount_amount, after_discount, vat_amount, final_total
END
```

ليش هي دقيقة وما فيها غموض؟ كل خطوة فيها عملية وحدة واضحة، وكل متغير له اسم يوضح وش فيه، والترتيب ثابت (ما فيه خطوة تستخدم قيمة قبل ما تنحسب)، وعدد الخطوات محدود وتخلص عند `END`. فلو أي شخص نفذها حرفيا بيطلع له نفس الناتج.

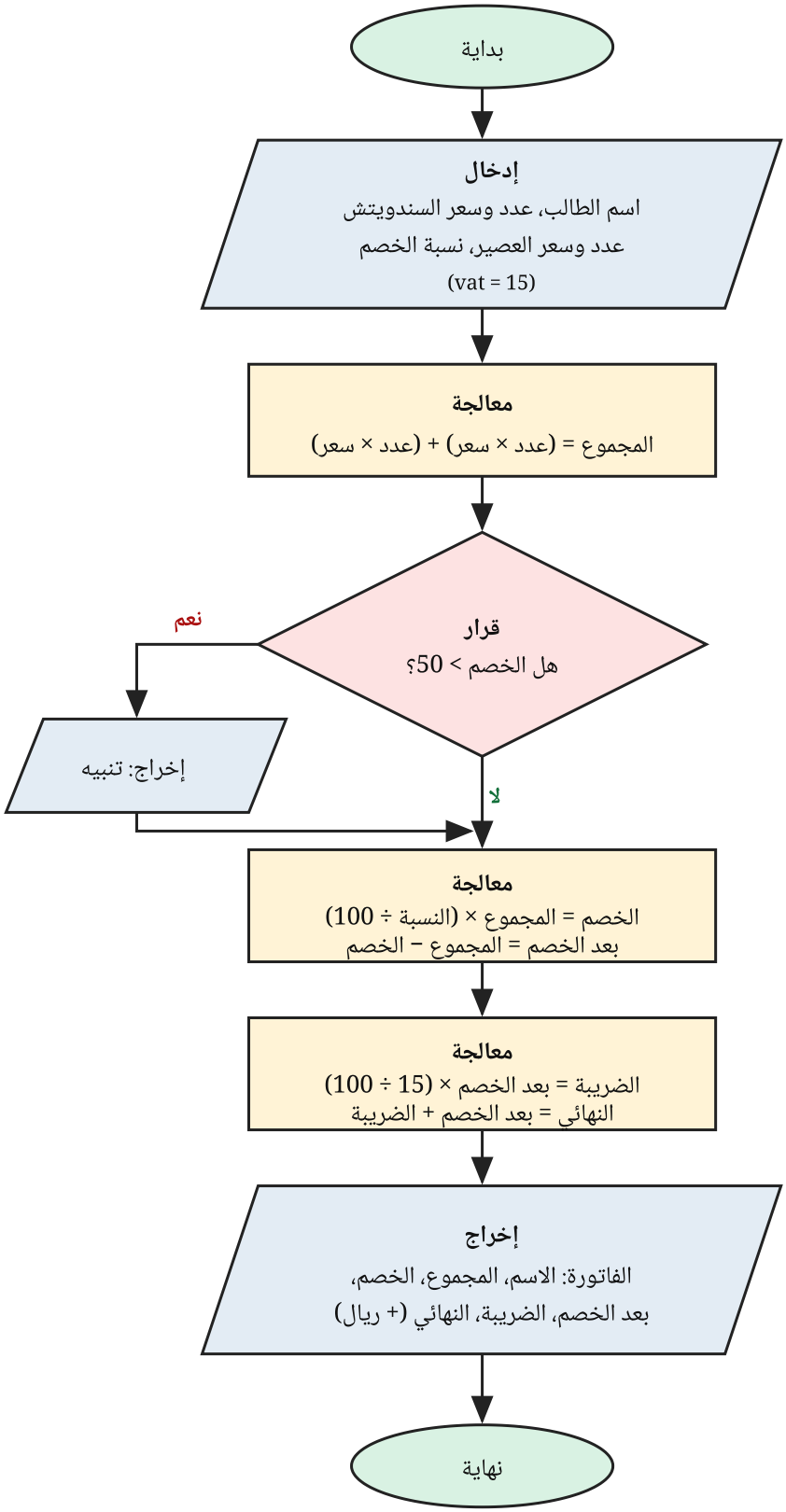
3) نفذتها بنفسي على مثال التحقق

المدخلات: 2 ساندويتش بـ 4.50، و 3 عصير بـ 2.00، وخصم 20%.

الخطوة	الحساب	الناتج
sandwiches_cost	`2 × 4.50`	9.00
juices_cost	`3 × 2.00`	6.00
subtotal	`9.00 + 6.00`	15.00
IF 20 > 50 ؟	لا	ما يطبع تنبيه
discount_amount	`15.00 × (20 ÷ 100)`	3.00
after_discount	`15.00 − 3.00`	12.00
vat_amount	`12.00 × (15 ÷ 100)`	1.80
final_total	`12.00 + 1.80`	**13.80**

نعم، وصلت لـ 13.80 ريال، نفس الناتج المطلوب.

الجزء ج: المخطط الانسيابي



الرموز: الشكل البيضاوي = بداية/نهاية، متوازي الأضلاع = إدخال/إخراج، المستطيل = معالجة، المعين = قرار.

3) المخطط والشيفرة الزائفة متطابقين

الترتيب	بالشيفرة الزائفة (الجزء ب)	بالمخطط
1	`START`	بداية
2	6 أوامر `INPUT` + `SET vat_percent = 15`	شكل الإدخال
3	`SET` المجموع	معالجة: المجموع
4	`IF discount_percent > 50` ← `OUTPUT` تنبيه	القرار + إخراج التنبيه
5	`SET` الخصم وبعد الخصم	معالجة: الخصم
6	`SET` الضريبة والنهائي	معالجة: الضريبة
7	`OUTPUT` الفاتورة	شكل الإخراج
8	`END`	نهاية

نفس الخطوات ونفس الترتيب. والقرار حطيته قبل حساب الخصم، وهذا نفس مكان التعليق `# TODO: warn if discount > 50` في البرنامج.

الجزء د: التتبع والكفاءة

1) جدول التتبع

الخطوة	السطر	a	b	المخرجات
1	`SET a = 5`	5	—	
2	`SET b = 3`	5	3	
3	`SET a = a + b`	8	3	
4	`SET b = a - b`	8	5	
5	`SET a = a - b`	3	5	
6	`OUTPUT a, b`	3	5	`3, 5`

وش تطبع؟ `3, 5`

وش سوت بالمتغيرين؟ بدلت قيمهم: `a` كانت 5 وصارت 3، و `b` كانت 3 وصارت 5. والحلو إنها بدلتهم بدون ما تحتاج متغير ثالث مؤقت، بس بالجمع والطرح.

2) البحث عن الرقم 32 في قائمة مرتبة من 1 إلى 32

طريقة «افحص الكل»: أبدأ من أول رقم وأفحص واحد واحد: 1، 2، 3، ... إلى 32. الرقم 32 آخر واحد، فأحتاج **32 خطوة** (هذي أسوأ حالة).

طريقة «احذف النص»: كل مرة أشوف الرقم اللي بالنص، ولأن 32 أكبر منه أحذف النص الأصغر:

الخطوة	الباقي	الرقم اللي بالنص	القرار
1	1 – 32	16	32 أكبر ← أحذف 1–16
2	17 – 32	24	32 أكبر ← أحذف 17–24
3	25 – 32	28	32 أكبر ← أحذف 25–28
4	29 – 32	30	32 أكبر ← أحذف 29–30
5	31 – 32	31	32 أكبر ← أحذف 31
6	32	32	لقيته

يعني تقريبا **6 خطوات** بس بدل 32 (القائمة تنقسم على 2 خمس مرات: 32 ← 16 ← 8 ← 4 ← 2 ← 1).

ليش «احذف النص» تحتاج قائمة مرتبة؟ لأني لما أشوف الرقم اللي بالنص وأقول «32 أكبر منه، فأكيد هو بالنص الثاني» هذا الكلام صحيح بس إذا الأرقام مرتبة. لو القائمة ملخبطة، ممكن يكون 32 بالنص اللي حذفته، فبحذفه بالغلط وما ألقاه.

3) أبعاد جودة الخوارزمية على خوارزمية الفاتورة

البعد	تقييمي لخوارزمية الفاتورة
الصحة	صحيحة: جربتها على مثال التحقق وطلعت 13.80 ريال بالضبط.
الوضوح	واضحة: أسماء المتغيرات تشرح نفسها (subtotal، vat_amount) وكل خطوة لها تعليق يربطها بالمشكلة الفرعية.
الكفاءة	كفؤة: عدد الخطوات ثابت (تقريبا 15 خطوة) مهما كانت الأرقام، وما فيها تكرار ما له داعي.
المتانة	ضعيفة شوي: لو المستخدم كتب كلمة بدل رقم يطلع خطأ ويوقف البرنامج، وتقبل أرقام سالبة. تحتاج تحقق من المدخلات.

الجزء و: الاختبار

شغلت البرنامج ثلاث مرات. هذي النتائج:

الحالة	المدخلات	المتوقع	اللي صار فعليا
طبيعية	مثال التحقق: 2 ،Faris ساندويتش بـ 4.50، 3 عصير بـ 2.00، خصم 20	`13.80 SAR`	`Total: 13.80 SAR` ✔ مطابق
حدية	2 ،Faris ساندويتش بـ 4.50، **0** عصير بـ 2.00، خصم **0**	المجموع 9.00، خصم 0.00، ضريبة 1.35، النهائي `10.35 SAR`	`Total: 10.35 SAR` ✔ مطابق (الصفر ما خرب شي)
غير صالحة	كتبت `three` بدل عدد السندويتشات	يطلع خطأ لأن `three` مو رقم	البرنامج وقف وطلع `ValueError` ✘ توقف

صورة التشغيلة 1 (طبيعية)

```
Student name: Faris
Number of sandwiches (whole number, e.g. 2): 2
Type before conversion: <class 'str'>
Type after conversion:  <class 'int'>
Sandwich price in SAR (e.g. 4.50): 4.50
Number of juices (whole number, e.g. 3): 3
Juice price in SAR (e.g. 2.00): 2.00
Discount percent (e.g. 10 means 10%): 20

----- Canteen Invoice -----
Student: Faris
Subtotal:            15.00 SAR
Discount (20%):      3.00 SAR
After discount:      12.00 SAR
VAT (15%):           1.80 SAR
Total:               13.80 SAR
---------------------------
```

صورة التشغيلة 2 (حدية)

```
Student name: Faris
Number of sandwiches (whole number, e.g. 2): 2
Type before conversion: <class 'str'>
Type after conversion:  <class 'int'>
Sandwich price in SAR (e.g. 4.50): 4.50
Number of juices (whole number, e.g. 3): 0
Juice price in SAR (e.g. 2.00): 2.00
Discount percent (e.g. 10 means 10%): 0

----- Canteen Invoice -----
Student: Faris
Subtotal:            9.00 SAR
Discount (0%):       0.00 SAR
After discount:      9.00 SAR
VAT (15%):           1.35 SAR
Total:               10.35 SAR
---------------------------
```

صورة التشغيلة 3 (غير صالحة)

```
Student name: Faris
Number of sandwiches (whole number, e.g. 2): three
Type before conversion: <class 'str'>
Traceback (most recent call last):
  File "تميم_الحارثي_الكلاس_بايثون.py", line 19, in <module>
    sandwich_count = int(sandwich_count_text)          # explicit int() for
counts
ValueError: invalid literal for int() with base 10: 'three'
```

شرح الحالة الثالثة

اسم الخطأ: `ValueError`، وهو خطأ وقت التشغيل (Runtime).

ليش طلع مع إن `input()` شغال؟ لأن `input()` فعلا اشتغل صح: هو دايم يرجع اللي كتبه المستخدم كنص (`str`)، وأخذ النص `"three"` بدون مشكلة (وهذا واضح من سطر `Type before conversion: <class 'str'>`). المشكلة صارت بالسطر اللي بعده: `int()` يحاول يحول النص لعدد صحيح، وهو يعرف يحول نص مثل `"3"` بس ما يفهم الكلمة الإنجليزية `"three"`، فيطلع `ValueError` ويوقف البرنامج.

الجزء ز: اصطياد الأخطاء

البرنامج 1: المفروض يرحب بالطالب

```
name = input("Your name: ")
print("Welcome, " name)
```

توقعي: ما بيشتغل أصلا، لأن بين النص `"Welcome, "` والمتغير `name` ما فيه `+` ولا فاصلة.

لما شغلته:

```
  File "b1.py", line 2
    print("Welcome, " name)
          ^^^^^^^^^^^^^^^^
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

التصليح:

```
name = input("Your name: ")
print("Welcome, " + name)
```

```
Your name: Tamim
Welcome, Tamim
```

البرنامج 2: المفروض يطبع عمر الطالب السنة الجاية

```
age = input("Your age in years: ")
next_year = age + 1
print(f"Next year you will be {next_year}")
```

توقعي: بيسأل عن العمر عادي، بعدين يطلع خطأ لأن `age` نص وأنا أجمع عليه رقم 1.

لما شغلته:

```
Your age in years: 16
Traceback (most recent call last):
  File "b2.py", line 2, in <module>
    next_year = age + 1
                ~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

التصليح: أحول المدخل لعدد صحيح بـ `int()`:

```
age = int(input("Your age in years: "))
next_year = age + 1
print(f"Next year you will be {next_year}")
```

```
Your age in years: 16
Next year you will be 17
```

البرنامج 3: المفروض يطبع SAR 54.00

```
price = 20
quantity = 3
discount = 10
total = price * quantity - discount / 100
print(f"Total after discount: {total:.2f} SAR")
```

توقعي: بيشتغل بدون أي رسالة خطأ، بس الرقم بيطلع غلط، لأن بايثون يسوي القسمة والضرب قبل الطرح، فيطرح 0.1 ريال بس بدل 10%.

لما شغلته:

```
Total after discount: 59.90 SAR
```

الحساب اللي صار: `60 - (10 / 100) = 60 - 0.1 = 59.90`، والصحيح: `60 - (60 × 10%) = 54.00`.

التصليح: أحسب الخصم من المجموع، وأحط أقواس:

```
price = 20
quantity = 3
discount = 10
total = (price * quantity) * (1 - discount / 100)
print(f"Total after discount: {total:.2f} SAR")
```

```
Total after discount: 54.00 SAR
```

سجل الأخطاء

البرنامج	نوع الخطأ	السبب	التصليح	الدليل (المخرجات بعد التصليح)
1	Syntax	ناقص `+` (أو فاصلة) بين النص والمتغير داخل `print`، فبايثون ما يقدر يقرا السطر أصلا.	`print("Welcome, " + name)`	`Welcome, Tamim`
2	Runtime	`input()` يرجع نص، والبرنامج حاول يجمع نص مع رقم ← `TypeError`.	`age = int(input(...))`	`Next year you will be 17`
3	Logic	ترتيب العمليات: القسمة صارت أول، فانطرح 0.1 بدل 10% من المجموع. البرنامج اشتغل بس الناتج غلط (59.90).	`total = (price * quantity) * (1 - discount / 100)`	`Total after discount: 54.00 SAR`

السؤال الأخير

قرار تصميم خذيته وليش: حطيت لكل خطوة حساب متغير وسيط خاص فيها (`subtotal`، `discount_amount`، `after_discount`، `vat_amount`، `final_total`) بدل ما أكتب كل الحساب بسطر واحد طويل. ليش؟ لأن كل متغير يطابق مشكلة فرعية من الجزء أ، فلو طلع رقم غلط بالفاتورة أعرف بسرعة أي خطوة فيها المشكلة، وكمان الفاتورة تحتاج تطبع كل هالقيم أصلا. وحطيت نسبة الضريبة بمتغير ثابت `VAT_RATE = 15` عشان لو تغيرت الضريبة أعدلها بمكان واحد.

تحسين أبغى أضيفه لو عندي وقت أكثر: أخلي البرنامج متين أكثر: لو المستخدم كتب كلمة مثل `three` أو رقم سالب، ما يوقف البرنامج، بل يطلع له رسالة واضحة ويطلب منه يكتب الرقم مرة ثانية. وأبرمج كمان التنبيه اللي محجوز بالتعليق `TODO` لما يكون الخصم أكبر من 50%.